An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary retains comparisons in directed acyclic graphs and represents possible orders by a region of real positions satisfying interval bounds and the recorded inequalities. It estimates the fraction of this region's volume retained by each answer through deterministic calculations on bounded graph neighborhoods. Moving items into ordered subproblems controls graph degrees and directed path lengths; a geometric analysis shows that these moves preserve the optimal asymptotic bound.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys.[1] A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them.

A binary comparison tree must distinguish all $n!$ total orders, giving the *information-theoretic lower bound* (ITLB) of $\lceil \log(n!) \rceil = n \log n - O(n)$ comparisons in the worst case [1, Section 5.3.1]. This bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [10].

[1] Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

Deterministic adversaries from graph entropy. Kahn and Kim [12] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Randomized approximation of the number of linear extensions. Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order P, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [9]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [11] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by P. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most ε, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most n^{-4}. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These

accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in n. Kaplan, Zamir, and Zwick [7, Section 7] ask whether an efficient adversary can attain this bound. We answer the following question in the affirmative (Theorem 7.1):

> *For every $\varepsilon > 0$, is there a deterministic polynomial-time sorting adversary that forces any correct sorting algorithm to perform at least $(1-\varepsilon)n \log n$ comparisons, for all sufficiently large n?*

Section 8 gives two further consequences. The same adversary forces nearly the information bound for arranging keys into a prescribed partial order, with a vanishing relative error and an additive $o(n)$ loss. A reduction also handles supplied initial relations: for an initial partial order P, it forces $(1-o(1)) \log e(P) - O(n \log \log n)$ further comparisons.

Section 2 reviews the earlier adversaries. Section 3 gives our data structure and comparison procedure. Items are stored at nodes of a rooted tree, with a directed acyclic graph recording comparisons within each node. Interval bounds and the graph's inequalities define a region of real positions representing the possible orders there. The adversary computes lower bounds on the fraction of this region's volume retained by each answer. Three technical ingredients make this possible.

Certifying comparisons with enclosing intervals. The stored interval bounds include the inequalities implied by the graph. We perform finite computations on a bounded neighborhood. Each computation returns intervals enclosing uniformly distributed feasible positions. Disjoint intervals establish a comparison, so counting these cases gives certified lower bounds on the probabilities of the two answers. Degree and path-length bounds ensure that almost all returned intervals are short; an unsuccessful certificate then implies that the queried positions are confined to very short intervals. This identifies an item to refine (Section 6).

Choosing positions with large remaining volume. Moving an item into a child fixes its position in the parent. We round to a binary fraction inside an interval chosen to ensure a large volume gain. For a structural repair, an interval with substantial probability inside and on both sides supplies the anchor: log-concavity gives a useful density bound throughout that interval. A failed certificate instead justifies using the middle half of the item's allowed interval (Section 5.1).

Showing that refinements are rare. Section 4 relates the volume gained by moving an item to the cost of choosing its child. Failed comparison certificates give large gains, as do complete batches of items moved from a directed path. High-degree moves are limited by the number of arcs inserted. Together, these facts show that only $O(q/k)$ refinements occur during q comparisons, where k is the degree threshold. Their total overhead is therefore small. A counting experiment combines node volumes and child choices to bound the number of orders still represented by the state.

Finally, Section 7 lets the degree and path-length bounds grow slowly with n, keeping the computation polynomial as the comparison coefficient approaches one. After $n \lceil \log n \rceil$ answers, the adversary can choose a fixed order consistent with its answers and use that order thereafter. This bounds the lifetime of the recursive construction.

The appendices supply the interval and density facts (Appendix A), quantitative certificate estimates (Appendices B and C), refinement and consistency proofs (Appendix D), and arithmetic details and parameter choices (Appendices E and F). Appendix G proves the two corollaries. The manuscript source is available online.[2]

[2] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

2 Related Work

2.1 Sorting with partial information

Fredman [2] studied how closely sorting can approach the information-theoretic lower bound when some orders have already been excluded. For any nonempty family Γ of permutations of n items, he showed that the unknown order can be identified using at most $\log|\Gamma| + 2n$ comparisons. In particular, if the available information is a partial order P, this gives $\log e(P) + 2n$ comparisons, where $e(P)$ counts its linear extensions. The comparison bound is within an additive $2n$ of the ITLB, but constructing the strategy requires enumerative information about the possible orders and does not give a polynomial-time algorithm for general partial orders [12, p. 390].

Kahn and Kim [12] later gave a deterministic polynomial-time sorting algorithm using $O(\log e(P))$ comparisons. They also constructed a deterministic polynomial-time adversary forcing $\Omega(\log e(P))$ comparisons. This history highlights the distinction between comparison complexity and the computation needed to choose comparisons. Our adversary faces the corresponding task of choosing answers efficiently while forcing asymptotically the full ITLB for ordinary sorting.

For an initial partial order P supplied as a DAG with m arcs, Haeupler et al. [15] give a deterministic algorithm using $O(m+\log e(P))$ time and $O(\log e(P))$ new comparisons. Rutschmann [17] separates this work into $O(m)$ preprocessing and $O(\log e(P))$ sorting time. These results make efficient use of the known relations; Corollary 8.2 supplies an efficient adversary whose comparison coefficient approaches one when $\log e(P) = \omega(n \log\log n)$.

Van der Hoog and Rutschmann [16] study a model with separate oracles for the initial partial order and the unknown total order. They establish tight tradeoffs between preprocessing queries and the later sorting cost. Our reduction processes the full supplied DAG; its guarantee concerns subsequent key comparisons. Lower bounds with restricted access to the initial relations require the additional analysis of their oracle model.

2.2 Binary-tree adversaries

Atallah and Kosaraju [3] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n\log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [4] on searching after partial sorting, Richards and Vaidya [5] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [6] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

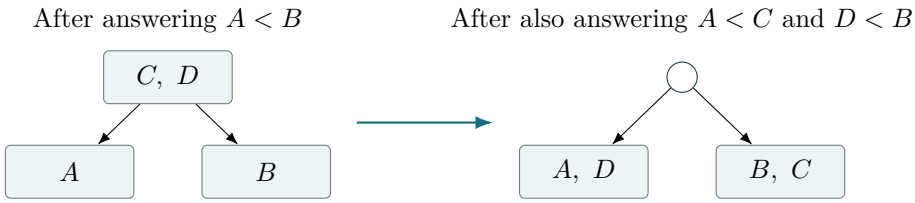


Figure 1: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2} n \log n$ comparisons are necessary [6, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

2.3 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [8] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i + 1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child i. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j - 1]$ or $[i + 1, j]$. Figure 2 shows this interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

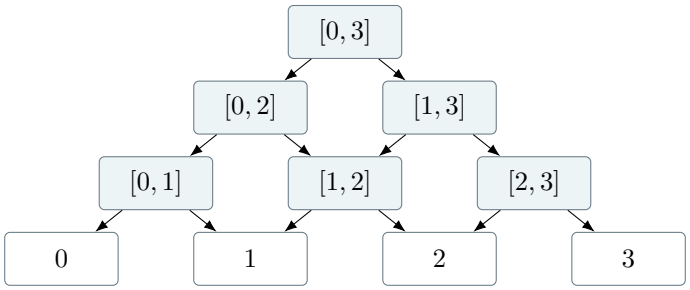


Figure 2: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [8, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree nodes and intermediate nodes representing three children may store pairs; intermediate nodes

representing two children store only unpaired items. Comparing two unpaired items A, B at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing A and C, one permitted response is $A < C$: move A into child 0 and B into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [8, Table 11].

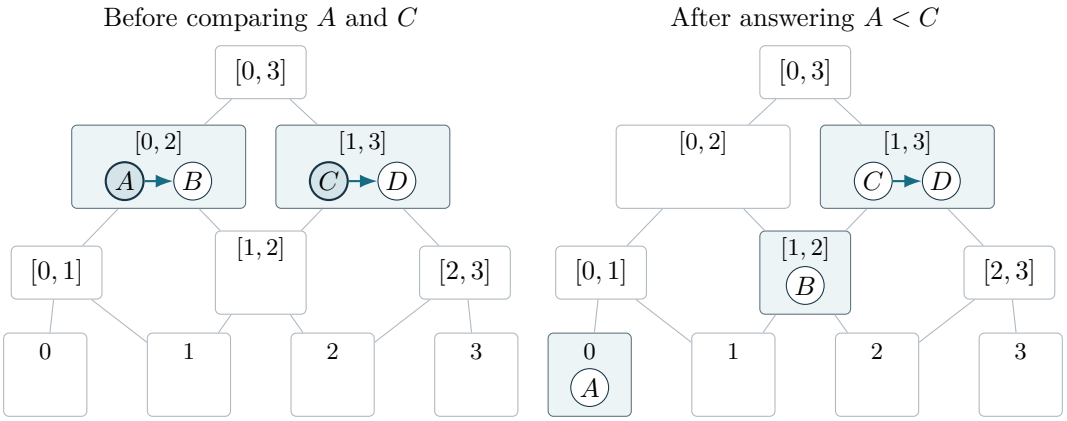


Figure 3: A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [8, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to A and C.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then A can enter only child 0 or 1, and B can enter only child 1 or 2. The pair's order thus narrows each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [8, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64} n \log n$ comparisons, improving the earlier $\frac{1}{2} n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [8, Section 6].

3 The new optimal adversary

We describe the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

Kaplan, Zamir, and Zwick [8] retain disjoint ordered pairs within a node. Our adversary maintains a rooted tree with a DAG at each node, so several retained comparisons can involve the same item. For example, it can keep $a < b$ and $a < c$ together while leaving the order of b and c undecided. This lets it represent richer partial orders before moving items further down the tree.

The computational challenge is to use these richer orders without having to approximately count the linear extensions of an entire DAG. We periodically simplify a node's DAG by removing vertices and moving their items into child nodes, creating children when needed. Constraints at the parent preserve the comparisons involving the moved items, while their order within a child can remain unresolved. These simplifications allow comparison answers to be chosen by deterministic calculations on small neighborhoods in that node's DAG. The analysis must show that the resulting loss of possible orders is small enough to retain the optimal comparison bound.

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. We call this its *in-node DAG*. Figure 4 shows an example.

Each item has a record (x, X_x, ℓ_x, r_x) stored at exactly one node. The label x identifies its key, and X_x is a symbolic variable for its position among the items at that node. The fields ℓ_x, r_x restrict this variable to the open interval $(\ell_x, r_x) \subseteq (0, 1)$. The bounds ℓ_x, r_x are fractions whose denominators are powers of two; we call such numbers *dyadic rationals*. We call an item *free* at its node while its position remains variable.

In an in-node DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. Each answered comparison between free items contributes at most one arc.

A *feasible assignment* at a node is a choice of real values for its position variables satisfying $\ell_x < X_x < r_x$ for every item x and $X_x < X_y$ for every arc $x \to y$ in its in-node DAG.

We keep each stored interval equal to the range of values its coordinate can take in feasible assignments. An arc $x \to y$ propagates lower bounds from x to y and upper bounds from y to x. After changing a node, the routine `Normalize` makes one pass in topological order for the lower bounds and one in reverse order for the upper bounds. Algorithm 4 gives the two passes; Appendix A proves that they preserve exactly the same feasible assignments.

Initially the tree consists of just the root, containing all records with bounds $\ell_x = 0$ and $r_x = 1$. The position variables X_x are unassigned, and the root's in-node DAG has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. A node has at most one child at each anchor. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position variables again range over $(0, 1)$, and its in-node DAG and interval bounds describe the possible orders inside this block.

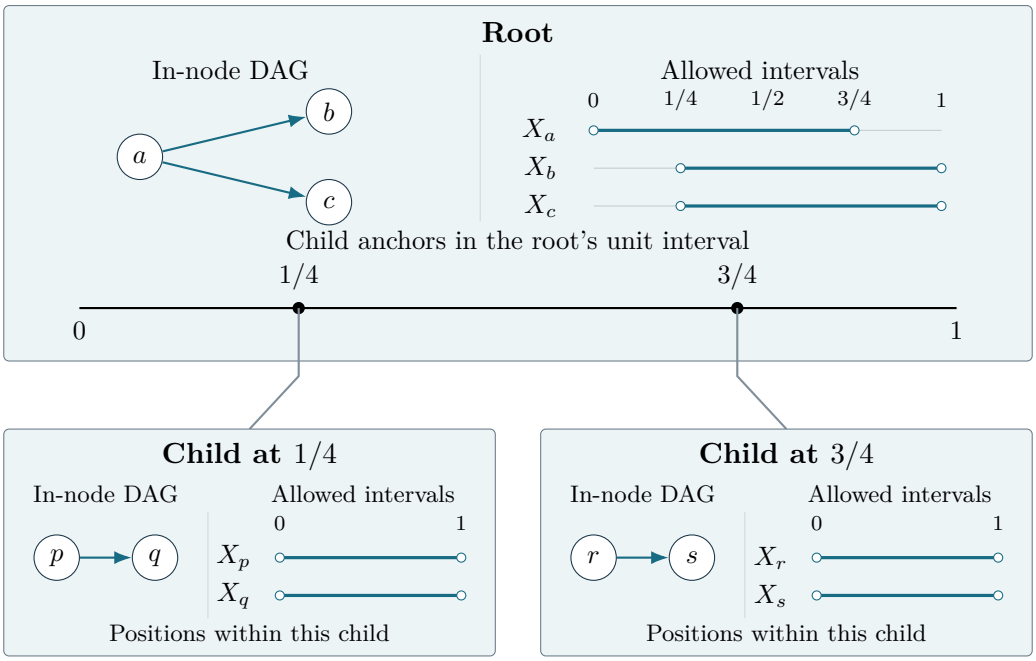


Figure 4: A state of the adversary. Each large box is a tree node, containing an in-node DAG and one open interval for each of its items.

In the root of Figure 4, the in-node DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. We call the sequence of these answers the *transcript*.

3.2 Answering a comparison

Algorithm 1 shows the algorithm the adversary uses to answer a comparison by the sorter.

To compare A and B, start at the root and descend into a child whenever its subtree contains both items. Stop when at least one item is stored at the current node, or when the items lie in different child subtrees. At this node, use an item's free coordinate if it is stored here, and the corresponding child anchor otherwise. In Figure 4, comparing p with q descends into the left child; comparing p with r uses the anchors $1/4$ and $3/4$; and comparing a with p compares X_a with $1/4$ at the root.

Two distinct anchors determine the answer immediately.

For a query involving a free coordinate, consider the uniform distribution on feasible positions at this node. The subroutine `Certify` computes two nonnegative rational numbers p_{AB}, p_{BA} that are lower bounds on the probabilities of $A < B$ and $B < A$. Fix a rational tolerance $0 < \eta < 1/32$. If the larger certified fraction is at least $1/2 - \eta$, choose that answer, breaking a tie in favor of $A < B$. Otherwise, move a queried free item into a child and restart the query. We call this move a *refinement*, denoted `Refine`(x, reason); Algorithm 3 in Section 3.3 gives the operation. Section 4 shows why a failed certificate makes such a refinement affordable. For two free items, the events are $X_A < X_B$ and $X_B < X_A$; for a free item and an anchor, they compare its coordinate with the fixed anchor. An answer between two free items adds an arc to the in-node DAG; an answer involving an anchor adds an interval bound.

After recording an answer, we repair its node if the new arc has made the in-node DAG too large. Fix integers $k, H \geq 2$. Between completed calls, every vertex has fewer than k neighbors and every directed path has fewer than H vertices. Degree counts both incoming and outgoing neighbors, and path length counts vertices. If a path of H vertices has appeared, refine all its items in path order. Otherwise, if a vertex has reached degree k, refine that vertex. Algorithm 2 gives this operation. One such repair restores both bounds: any new violation must involve the arc just inserted, and the repair removes that arc. Section 5.2 proves this claim.

The following lemma supplies the guarantee needed by the comparison rule. Section 6 implements `Certify` by counting finite calculations in which enclosing intervals establish one of the two answers.

Lemma 3.1 (Certified comparison probabilities)**.** *Suppose a node admits a feasible assignment, and its in-node DAG has degree at most k and directed paths of at most $2H$ vertices. For two distinct free coordinates, or a free coordinate and a dyadic threshold,* `Certify` *deterministically returns nonnegative rational lower bounds on the probabilities of the two answers.*

The recursive construction is needed for at most

$$Q(n) = n \lceil \log n \rceil \tag{1}$$

answers. Record these answers in a list $\mathcal{T}$, initially empty. If another query arrives after $Q(n)$ answers, choose a fixed total order consistent with $\mathcal{T}$ and use it for every subsequent answer. Section 7 constructs this order by a deterministic topological sort. A sorter reaching this point has already made at least $n \log n$ comparisons.

Algorithm 1: `Compare`(A, B): answer a comparison query.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.
if *fixed ranks have been chosen* **then**
  return their order of A, B
if $|\mathcal{T}| = Q(n)$ **then**
  Form the DAG of answers in $\mathcal{T}$ and choose a topological order, breaking ties by label
  Store its ranks and **return** their order of A, B
while *true* **do**
  $v \leftarrow$ the root
  while *A and B lie in the same child subtree of v* **do**
    $v \leftarrow$ that child
  if *A and B lie in different child subtrees of v* **then**
    answer $\leftarrow$ the left-to-right order of their child anchors
    break
  $(p_{AB}, p_{BA}) \leftarrow$ `Certify`(v, A, B, η)
  if $\max(p_{AB}, p_{BA}) \geq 1/2 - \eta$ **then**
    answer $\leftarrow A < B$ if $p_{AB} \geq p_{BA}$, and $B < A$ otherwise
    break
  $x \leftarrow$ the selected queried free item
  `Refine`$(x, \text{failure})$
if *at least one queried item is stored at v* **then**
  Store *answer* as an arc in the in-node DAG at v or as an interval bound
  `Normalize`(v)
Append *answer* to $\mathcal{T}$
`Repair`(v)
return *answer*

Before recording the answer, only the two queried items can move. Section 5.2 shows that each moves at most once in this part of the call. Thus the loop makes at most two refinements and three comparison-certificate calculations. A queried item may move again in the subsequent repair; the whole call makes at most $n + 2$ refinements.

For deterministic selection, choose the smallest eligible item label and the lexicographically first eligible directed path. Complete the entire selected path batch, even if earlier refinements have already removed its arcs.

Algorithm 2: `Repair`(v): restore the in-node DAG bounds after an answer.

Input: The node v at which the answer was determined.
if *the in-node DAG at v has a directed path on H vertices* **then**
  $L \leftarrow$ the selected path's list of labels, in path order
  foreach $x \in L$ *in path order* **do**
    `Refine`(x, repair)
else if *a vertex at v has degree k* **then**
  $x \leftarrow$ the selected such vertex
  `Refine`(x, repair)

3.3 Moving an item into a child

Algorithm 3 gives `Refine`(x, reason), which moves x from its current node into a child. The argument *reason* records whether the move follows a failed comparison certificate or is part of a structural repair.

After a failed comparison certificate, choose an anchor in the middle half of (ℓ_x, r_x). For a structural repair, first try to find an interval with substantial probability inside and on both sides. If that calculation fails, use the middle half instead. In either case, a simple rounding rule selects a dyadic position in the chosen interval; Algorithm 5 in Section 5.1 gives the rule.

Lemma 3.2 (A central interval)**.** *Under the hypotheses of Lemma 3.1,* `CentralInterval` *deterministically returns either failure or dyadic $u < v$ in (ℓ_x, r_x) satisfying*

$$\mathbb{P}(X_x \le u) \ge \frac{1}{4}, \qquad \mathbb{P}(u \le X_x \le v) \ge \frac{1}{4}, \qquad \mathbb{P}(X_x \ge v) \ge \frac{1}{4}.$$

Section 6 implements this routine using the same enclosing intervals as `Certify`.

Fixing $X_x = c$ places x in the child block anchored at c. The arcs incident to x in that in-node DAG become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, and retain the new bounds on its neighbors. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh position variable with interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 5 illustrates reuse of an existing child.

Algorithm 3: `Refine`(x, reason): move an item into a child.

Input: A free item x, the reason for its move, and the current adversary state.
Output: The updated state, with x stored in a child of its former node.
$v \leftarrow$ the node storing x
$c \leftarrow$ `ChooseAnchor`(x, reason)
foreach *arc* $y \to x$ *in the in-node DAG at* v **do**
  $r_y \leftarrow \min(r_y, c)$
foreach *arc* $x \to z$ *in the in-node DAG at* v **do**
  $\ell_z \leftarrow \max(\ell_z, c)$
Remove x's record from v and its vertex and incident arcs from the in-node DAG at v
`Normalize`(v)
if v *has no child anchored at* c **then**
  Create a child of v at c, with no records and an empty in-node DAG
$u \leftarrow$ the child of v anchored at c
Insert a record $(x, X_x, 0, 1)$ at u, with a fresh, unassigned position variable X_x
Insert x as an isolated vertex in the in-node DAG at u

The propagation routine used above keeps the stored intervals up to date. For example, tightening r_y also tightens the upper bound of every predecessor of y. The two passes below perform all such propagations.

Algorithm 4: `Normalize`(v): propagate interval bounds through the in-node DAG.

Input: A node v whose arcs or interval bounds have changed.
foreach *vertex* x *in topological order* **do**
  $\ell_x \leftarrow \max(\{\ell_x\} \cup \{\ell_y : y \to x\})$
foreach *vertex* x *in reverse topological order* **do**
  $r_x \leftarrow \min(\{r_x\} \cup \{r_y : x \to y\})$

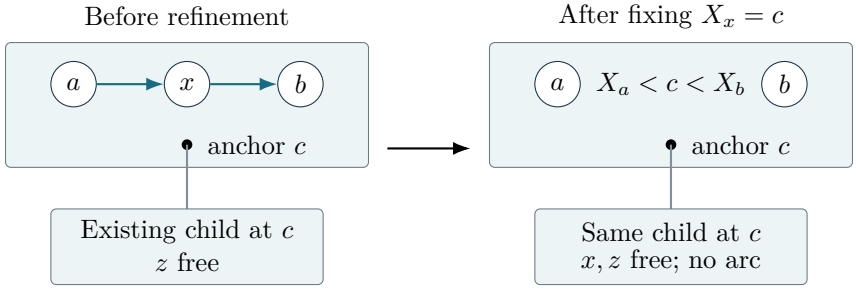


Figure 5: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of x within that block stays open. Lemma 5.3 proves that this operation preserves all answered comparisons.

Refinement can impose additional order relations. For example, suppose the root contains A, B, C with only $A < B$ recorded. Refining A at $1/2$ restricts B to $(1/2, 1)$. Refining C into the same child then leaves both $A < C < B$ and $C < A < B$ possible, while fixing the additional relation $C < B$. Section 4 shows that we can afford this loss of possible orders while retaining the optimal asymptotic comparison bound.

Table 1 collects the thresholds used by Algorithms 1–4 above. In the next section, we explain why the adversary forces any correct sorter to perform many comparisons.

4 Why the adversary forces many comparisons

Recall from Algorithm 1 that a larger degree threshold k lets the adversary retain more comparisons involving each item before moving it into a child. Increasing k lets the adversary force a comparison count closer to $n \log n$. The following bound quantifies this dependence.

Theorem 4.1 (Comparison bound)**.** *For every sufficiently large integer k, the adversary of Algorithm 1, with the remaining parameter choices in Table 1, is deterministic and legal, and every call terminates. Every correct sorter makes at least*

$$\frac{\log(n!) - n}{1 + O(\log k/k)} \tag{2}$$

comparisons.

For this section, fix a sufficiently large integer k. Section 7 chooses k as a function of n so that it grows without bound while the adversary's running time remains polynomial.

We will bound the number of total orders represented by any state. The bound combines the freedom within nodes with weights assigned to child choices. We then show that refinements are rare: after q completed calls, there have been only $O(q/k)$ moves. This makes their total contribution to the comparison bound small.

4.1 Volume changes

We first measure the freedom left within the individual nodes. At a node v, the feasible assignments form the region

$$K_v = \{X : \ell_x < X_x < r_x \text{ for every free } x, \quad X_x < X_y \text{ for every in-node arc } x \to y\}. \tag{3}$$

Write $V_v = \mathrm{vol}(K_v)$ and take the product over the adversary's tree:

$$Z = \prod_v V_v. \tag{4}$$

Each item has a position variable at exactly one node, so the product region has dimension n. A node with no free items contributes volume one. Initially $Z = 1$; whenever every node has positive volume, $0 < Z \leq 1$.

Choose the certificate tolerance to be

$$\eta = \frac{1}{100k}. \tag{5}$$

By Lemma 3.1 and the acceptance test in Algorithm 1, each chosen answer retains at least $1/2 - \eta$ of its node's volume. Its decrease in $\log Z$ is therefore at most

$$a_\eta = -\log(1/2 - \eta) = 1 + O(1/k). \tag{6}$$

A comparison between two child anchors leaves Z unchanged.

Refinement fixes an item's position in its parent and gives it a fresh unit coordinate in a child. Write

$$g = \log \frac{Z_{\text{after}}}{Z_{\text{before}}} \tag{7}$$

for the resulting change in $\log Z$. We call g the refinement's *volume gain*; it can be negative.

To evaluate this gain for an item x stored at node v, let $S_x(c)$ be the volume of the section of K_v obtained by fixing $X_x = c$, measured in the remaining coordinates. Under the uniform distribution on K_v, the coordinate X_x has density

$$f_x(c) = \frac{S_x(c)}{V_v}, \qquad \mathbb{P}(a < X_x < b) = \int_a^b f_x(t)\, dt.$$

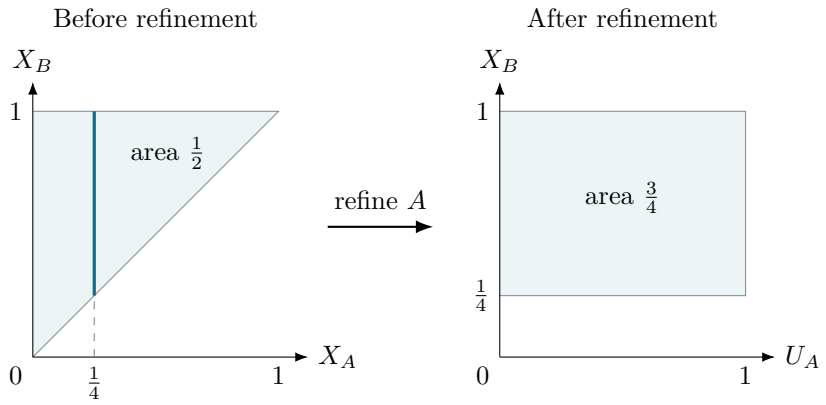


Figure 6: The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

Lemma 4.2 (Refinement identity)**.** *Suppose every node region has positive volume. Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{8}$$

Proof. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0, 1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0, 1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. Normalizing the interval bounds preserves these regions. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

Thus the refinement's volume gain is $g = \log f_x(c)$.

For example, the parent region $0 < X_A < X_B < 1$ has area 1/2. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length 3/4. The new child coordinate $U_A \in (0, 1)$ turns this into a rectangle of area 3/4, so refinement multiplies Z by 3/2; see Figure 6. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

The algorithm must preserve positive node volumes and the bounds needed by the probability routines. It must also return answers consistent with the earlier comparisons. The following proposition gives these guarantees.

Proposition 4.3 (Consistency and termination)**.** *During the recursive phase, every reached state has positive node volumes and represents only orders consistent with the transcript. Each in-node DAG has degree at most k and directed paths of fewer than $2H$ vertices. Between completed calls, the stronger bounds hold: every degree is less than k and every directed path has fewer than H vertices. A call performs at most two refinements before recording its answer and at most $n + 2$ refinements in total, and always terminates.*

Section 5.2 explains the repair and progress arguments; Appendix D proves the full execution guarantee.

4.2 Counting the orders represented by a state

The product Z measures freedom within individual nodes. For example, two items stored alone in separate children can each retain the full interval $(0, 1)$, giving $Z = 1$, even though the child anchors fix their order. To account for this restriction, assign a positive weight $w(c)$ to each child at anchor c, with the weights at each parent summing to at most $1/2$.

For each item, multiply the child weights along the path from the root to its storage node. Let $\mathcal{W}$ be their product over all items. An item stored at the root contributes one. Thus each edge weight appears once for every item in its subtree.

Lemma 4.4 (Number of represented orders)**.** *If the state represents N_{ord} total orders, then*

$$N_{\text{ord}} \geq n!\, 2^{-n} Z \mathcal{W}. \tag{9}$$

Proof. On the fixed state tree, start at the root and enter each existing child c with probability $w(c)$. With the remaining probability, stop and choose a uniform coordinate in $(0, 1)$ at the current node. The stopping probability is at least $1/2$, and the finite tree ensures termination.

Draw one such position independently for every label. The state tree orders these positions by comparing free coordinates and child anchors, then ordering positions within a child recursively. Ties have probability zero. All labels have the same distribution, so each of the $n!$ relative orders has probability $1/n!$.

Consider the event that every label stops at its actual storage node and the coordinates satisfy all in-node constraints. The required paths have probability $\mathcal{W}$, and stopping at their endpoints contributes at least 2^{-n}. Conditional on these choices, the coordinates are independent and uniform, so the probability of satisfying the constraints is Z. Every order produced by this event is represented by the state. Its probability is therefore at most $N_{\text{ord}}/n!$, proving the lemma. □

Define the cost of choosing a child by

$$\ell(c) = -\log w(c). \tag{10}$$

Records only move downward. Their current paths therefore contain exactly the edge traversals made during refinement, counted once for each moved item. Consequently

$$-\log \mathcal{W} = \sum_{\text{refinements } r} \ell(c_r). \tag{11}$$

4.3 One bound for the cost of refinement

We now choose the child weights so that their costs can be compared with the volume changes caused by refinement. Write a dyadic anchor in reduced form as

$$c = \frac{2i+1}{2^{m+1}}, \qquad m \geq 0, \quad 0 \leq i < 2^m, \tag{12}$$

where i, m are integers, and assign it weight

$$w(c) = \frac{1}{2k} \left(\frac{1 - 1/k}{2} \right)^m. \tag{13}$$

There are 2^m anchors with a given exponent $m + 1$, so their total weight is

$$\sum_c w(c) = \frac{1}{2k} \sum_{m \geq 0} (1 - 1/k)^m = \frac{1}{2}.$$

In particular, the existing children at any node satisfy the weight condition of Lemma 4.4. The cost is

$$\ell(c) = \gamma m + \log k + 1, \qquad \gamma = 1 - \log(1 - 1/k). \tag{14}$$

The anchor routine will provide the following guarantee. It says that choosing a child costs approximately the volume gained by fixing the parent coordinate, with an additive term of order $\log k$.

Lemma 4.5 (Cost and volume change of a refinement)**.** *There are coefficients $a \geq 1$ and $b \geq 0$, with $a = 1 + O(1/k)$ and $b = O(\log k)$, such that every refinement satisfies*

$$\ell(c) \leq ag + b. \tag{15}$$

A refinement made after unsuccessful comparison certification has gain $g > k$.

Appendix D.1 proves the lemma. We will use it to show that the additive term b is incurred only rarely. Choose

$$H = 2^k. \tag{16}$$

4.4 Refinements are rare

Lemma 4.6 (Refinement count)**.** *After q completed calls in the recursive phase, the total number P of refinements is $O(q/k)$.*

Proof. Let D count degree repairs. Each deletes k in-node arcs, while each answer inserts at most one arc and each inserted arc is deleted at most once. Hence $D \leq q/k$.

The other refinements contribute large volume gains. A move after failed comparison certification has gain greater than k. For a path batch, write its labels in path order as $x_1, \ldots, x_H$ and their chosen anchors as $c_1, \ldots, c_H$. Refining x_i turns its arc to x_{i+1} into the bound $X_{x_{i+1}} > c_i$. The next interior anchor therefore satisfies $c_{i+1} > c_i$. The anchors are distinct, and their weights sum to at most $1/2$. By the arithmetic–geometric mean inequality,

$$\prod_{i=1}^{H} w(c_i) \leq \left(\frac{1}{2H}\right)^H, \qquad \sum_{i=1}^{H} \ell(c_i) \geq H \log(2H) = H(k+1).$$

Together with (15), this gives

$$\sum_{i=1}^{H} g_i \geq H \frac{k + 1 - b}{a} \geq \frac{kH}{2}$$

for $k \geq 64$, using the coefficient bounds proved in Appendix D.1. Each completed call finishes its entire selected path batch, so all $P - D$ moves outside degree repairs contribute at least $k/2$ per move on average.

Degree repairs may have negative gain. Since $\ell(c) \geq 0$, (15) gives $g \geq -b/a$ for every move. Thus the total refinement gain $G = \sum_r g_r$ satisfies

$$G \geq \frac{k}{2}(P - D) - \frac{b}{a} D.$$

Comparisons each decrease $\log Z$ by at most a_η, and initially $Z = 1$. Therefore

$$\sum_r g_r \leq \log Z + a_\eta q \leq a_\eta q. \tag{17}$$

Combining these inequalities gives

$$\frac{k}{2}(P - D) \leq a_\eta q + \frac{b}{a} D.$$

Now $D \leq q/k$, $a_\eta = O(1)$, and $b/a = O(\log k)$, so $P = O(q/k)$. □

4.5 The comparison bound

Proof of Theorem 4.1. Proposition 4.3 gives consistency and termination during the recursive phase. At the cutoff, at least one total order respects all recorded answers, so their directed graph is acyclic. Its chosen topological order extends these answers and makes every later answer consistent with them. All procedures and tie rules are deterministic.

If the sorter reaches the cutoff $Q(n)$ in (1), it has already made at least $n \log n$ comparisons, which suffices for the claimed bound. Otherwise it finishes after $q < Q(n)$ calls to the recursive procedure. By Lemma 4.6, these completed calls perform $P = O(q/k)$ refinements. Summing (15) and using (17) gives

$$\sum_r \ell(c_r) \le a \sum_r g_r + bP \le a \log Z + a a_\eta q + bP.$$

Combining this with the remaining-order bound (9) and (11) yields

$$\log N_{\mathrm{ord}} \ge \log(n!) - n + (1-a)\log Z - a a_\eta q - bP. \tag{18}$$

The term $(1-a)\log Z$ is nonnegative. Also,

$$a a_\eta = 1 + O(1/k), \qquad bP = O\left(\frac{q \log k}{k}\right).$$

Consequently

$$\log N_{\mathrm{ord}} \ge \log(n!) - n - (1 + O(\log k/k))\, q.$$

Thus many orders remain until nearly $\log(n!)$ comparisons have been made. At termination $N_{\mathrm{ord}} = 1$, giving

$$q \ge \frac{\log(n!) - n}{1 + O(\log k/k)}.$$

This proves (2). □

The next section gives the rounding rule and explains why one repair restores the in-node DAG bounds. Appendix D supplies the full refinement and consistency proofs.

5 Choosing anchors and repairing nodes

We complete the refinement rule by specifying how to choose an anchor. We then explain why one structural repair after each answer restores the in-node DAG bounds, and why selecting an answer requires at most two moves.

5.1 Choosing an anchor

An anchor should lie where fixing the coordinate retains a substantial section of the feasible region. A short binary representation also keeps its child-choice cost small. We obtain both properties by choosing an interval and rounding one of its endpoints.

For a nondegenerate closed interval $J = [u, v] \subset (0, 1)$, choose the largest power-of-two spacing h no greater than its length. The routine `RoundUp` rounds u up to a multiple of h:

$$h = 2^{-r}, \qquad h \le |J| < 2h, \qquad c = h\lceil u/h \rceil. \tag{19}$$

Lemma 5.1 (Rounding to a binary fraction)**.** *The value c returned by* `RoundUp`(J) *belongs to J. Writing $c = (2i+1)/2^{m+1}$ in reduced form, it satisfies*

$$|J| < 2^{-m}. \tag{20}$$

Proof. The rounded value obeys $u \le c < u + h \le v$, so $c \in J$. The choice of spacing gives $h \le |J| < 2h$. Reduction of the fraction cannot increase its denominator, so $m + 1 \le r$ and $|J| < 2h \le 2^{-m}$. □

The reason for refinement determines the interval used by Algorithm 5. After failed comparison certification, it uses the middle half of the coordinate's interval directly. For a structural repair it first asks `CentralInterval` for an interval with substantial probability on both sides and inside; if this calculation fails, it also uses the middle half.

Algorithm 5: `ChooseAnchor`(x, reason): choose a position for refinement.

Input: A free item x at node v, with stored interval (ℓ_x, r_x); $\mathit{reason} \in \{\text{failure}, \text{repair}\}$.
Output: A dyadic anchor $c \in (\ell_x, r_x)$.
$J \leftarrow$ `FAIL`
if $\mathit{reason} =$ repair **then**
  $J \leftarrow$ `CentralInterval`(v, x)
if J *is* `FAIL` **then**
  $W_x \leftarrow r_x - \ell_x$
  $J \leftarrow [\ell_x + W_x/4,\ r_x - W_x/4]$
return `RoundUp`(J)

The successful central choice explains the relation between interval length and volume gain. We use the following geometric fact, proved in Appendix A.

Lemma 5.2 (Density in a central interval)**.** *Let X be uniform on a positive-volume node region. Suppose $J = [u, v] \subset (\ell_x, r_x)$ satisfies*

$$\mathbb{P}(X_x \le u) \ge \tfrac{1}{4}, \qquad \mathbb{P}(X_x \in J) \ge \tfrac{1}{4}, \qquad \mathbb{P}(X_x \ge v) \ge \tfrac{1}{4}.$$

Then, for every $c \in J$,

$$f_x(c) \ge \frac{1}{16|J|}.$$

When `CentralInterval` succeeds, its guarantee supplies these three mass bounds. Lemma 5.1 and the refinement identity therefore give

$$g = \log f_x(c) \ge \log(1/|J|) - 4 > m - 4. \tag{21}$$

Thus a longer binary representation is accompanied by a larger volume gain. Appendix D.1 proves the corresponding bound for the middle-half choice and deduces the affine cost guarantee of Lemma 4.5.

5.2 One repair after each answer

An interior anchor allows an item to move without contradicting earlier answers, even when its destination child already contains items. We state this property here; Appendix D proves it by examining the orders represented before and after the move.

Lemma 5.3 (Refinement and child admission)**.** *Suppose every node has positive volume and every represented total order respects the transcript. Refining an item at an interior anchor preserves both properties, including when its destination child already exists.*

Between calls, the adversary maintains degree below k and directed paths with fewer than H vertices. The important feature of Algorithm 2 is that every new violation involves the arc just inserted.

Lemma 5.4 (One-arc repair)**.** *Suppose all in-node DAGs have degree below k and directed paths with fewer than H vertices before an answer is recorded. Recording the answer and executing* `Repair` *restores these bounds. During this operation, degrees are at most k and directed paths have fewer than $2H$ vertices.*

Proof. An answer involving an anchor adds only an interval bound. An answer between free items adds at most one arc, say $x \to y$. Its certified positive probability ensures that the resulting region is nonempty, so the arc cannot create a directed cycle. Only x and y can reach degree k. Every new path consists of two old paths joined by this arc, so it has fewer than $2H$ vertices.

If an H-vertex path now exists, it must contain $x \to y$. Refining its entire vertex set removes both endpoints. Otherwise a degree repair removes an endpoint that has reached degree k. Either repair deletes the new arc; the remaining parent DAG is a subgraph of the previously valid DAG. Each child receives only isolated vertices, so its bounds remain valid too. If neither violation occurs, no repair is needed. □

Before the answer is recorded, only the queried labels can move. Once one has been refined, its fresh coordinate is isolated with interval $(0, 1)$. Further refinements leave it isolated and leave this interval unchanged. Refining it again at that stage would have volume gain zero, by the refinement identity. Lemma 4.5, however, requires gain greater than k after failed comparison certification. Thus each queried label moves at most once before an answer, giving at most two refinements and three comparison-certificate calculations.

After the answer, a queried label may move again in the structural repair. The repair refines either one vertex or a whole path of at most n vertices. Hence a full call makes at most $n + 2$ refinements. Appendix D combines these observations with the consistency argument to prove Proposition 4.3.

6 Computing comparison certificates

We implement `Certify` and `CentralInterval` by computing finite lists of enclosing intervals for the queried positions. Disjoint intervals certify a comparison. Counts of intervals lying to the left, inside, and to the right of a proposed central interval certify its three probability bounds.

6.1 A coordinate update with finitely many choices

Let G be the in-node DAG, with its stored canonical interval endpoints ℓ_z, r_z. For a current assignment Y, the interval permitted at coordinate z has endpoints

$$L_z = \max\big(\{\ell_z\} \cup \{Y_u : u \to z\}\big), \qquad U_z = \min\big(\{r_z\} \cup \{Y_u : z \to u\}\big).$$

Fix a power of two S. For a digit $j \in \{0, \dots, S-1\}$, update

$$Y_z \leftarrow \frac{Y_z + (S-1-j)L_z + jU_z}{S}. \tag{22}$$

The coefficients are nonnegative and sum to one. Thus an assignment satisfying the interval bounds and arc inequalities weakly continues to do so after an update, including when $L_z = U_z$.

Choosing the digit uniformly preserves the uniform distribution on the feasible region. Start from a uniform feasible assignment and condition on all coordinates other than z. Then Y_z is uniform on (L_z, U_z). Its normalized position $r = (Y_z - L_z)/(U_z - L_z)$ is sent to

$$r' = \frac{r + j}{S}.$$

For a fixed digit, this is uniform on the corresponding one of the S equal subintervals. Averaging over the digits makes r' uniform on $(0, 1)$ again.

6.2 Enclosing intervals from two calculations

Fix a topological order of the full graph G, and let T be the set of queried coordinates. Retain every vertex within undirected distance ρ of T, together with all arcs between retained vertices. Retain their stored interval endpoints. Vertices at distance exactly ρ form the boundary; those at smaller distance form the interior.

Run two calculations using the same digits. The lower calculation starts at $A_z = \ell_z$, and the upper calculation starts at $B_z = r_z$. Keep the boundary values fixed throughout both calculations. A *sweep* updates each interior coordinate once, in the restriction of the full topological order. All neighbors of an updated vertex are retained.

Enumerate every assignment of digits to t sweeps. If there are b' interior vertices, this gives

$$N = S^{b't} \tag{23}$$

computations. Each contributes the intervals $[A_x, B_x]$, $x \in T$, to the list, including repeated entries.

Algorithm 6: `Enclose`: compute a finite list of enclosing intervals.

Input: The retained in-node DAG and its stored interval bounds, its interior vertices, the restricted full topological order, queried coordinates T, and integers S, t.
Output: A list of enclosing intervals $([A_x, B_x])_{x \in T}$, one per digit assignment.

```
L ← the empty list
foreach assignment of digits j_{z,s} ∈ {0, ..., S − 1} to the b't interior visits do
    A_z ← ℓ_z and B_z ← r_z for every retained vertex z
    for s = 1, ..., t do
        foreach interior vertex z in the supplied order do
            Apply (22) to A_z, using the current A values and digit j_{z,s}
            Apply (22) to B_z, using the current B values and the same digit
    Append ([A_x, B_x])_{x∈T} to L
return L
```

The intervals have an exact probabilistic interpretation. A *coupling* is a joint distribution with specified marginal distributions; here it joins a uniformly selected list entry to a uniform feasible assignment.

Theorem 6.1 (Local enclosing intervals)**.** *Let $d, h \geq 1$ be integers and let $\delta, \alpha \in (0,1)$ be rational. There are computable parameters S, t, ρ and a bound E, depending only on d, h, δ, α, with the following property.*

Suppose G is a finite DAG with canonical dyadic interval bounds, degree at most d, directed paths of at most h vertices, and a positive-volume feasible region $K \subseteq (0,1)^{V(G)}$. For any nonempty set T of one or two coordinates, Algorithm 6 returns a list satisfying two guarantees:

1. *A uniformly selected entry can be coupled with X uniform on K so that $A_x \leq X_x \leq B_x$ for every $x \in T$.*
2. *At most a fraction α of its entries have $\max_{x \in T}(B_x - A_x) > \delta$.*

The calculation enumerates at most E digit assignments. If the stored bounds have at most Λ fractional binary digits, every computed endpoint has at most $\Lambda + \log E$ fractional digits.

The enclosure guarantee holds for every finite number of sweeps and every choice of radius $\rho \geq 1$. Start a full assignment uniformly on K and update the entire graph with independent uniform digits. Share its digits with the two endpoint calculations at interior vertices. The update is increasing in every coordinate, since maxima, minima, and the nonnegative linear combination in (22) are increasing. Thus it preserves the inequalities $A_z \leq X_z \leq B_z$ at an

interior update. At a boundary vertex the full position always remains between its fixed interval endpoints, even as it changes during the full calculation. The full assignment remains uniform, and every interior digit sequence has probability $1/N$. This proves the first guarantee.

To make most intervals narrow, we use the degree and path-length bounds. Following dependencies on newly updated predecessors reaches a source within h vertices, where the lower endpoint is fixed. Most digit choices therefore reduce the interval widths. Bounded degree limits the number of dependencies to consider across successive sweeps. Appendix B proves the width bound and gives explicit parameter choices. Appendix E proves the precision bound.

6.3 Certifying comparison answers

For two queried coordinates, let p be the fraction of entries with $B_x < A_y$, and let q be the fraction with $B_y < A_x$. The routine `Certify` always returns these two counts divided by N. For a threshold comparison, use the fixed interval $[c, c]$ for the threshold. The comparison rule accepts an answer when its certified fraction is at least $1/2 - \eta$; otherwise it refines a queried free item.

Proof of Lemma 3.1. The two certification conditions are disjoint, so $p, q \geq 0$ and $p + q \leq 1$. Whenever $B_x < A_y$, enclosure gives $X_x < X_y$; the reverse condition gives the reverse order. Consequently

$$p \leq \mathbb{P}(X_x < X_y), \qquad q \leq \mathbb{P}(X_y < X_x).$$

The same argument applies to a fixed threshold. These bounds hold for every list, including when neither fraction passes the acceptance test. The counts and their denominators are exact integers, so the routine is deterministic. □

6.4 Certifying a central interval

For a free coordinate x, `CentralInterval` uses the one-coordinate list. It chooses a candidate interval $[u, v]$ from the sorted endpoints and checks how many list intervals lie to its left, wholly inside it, and to its right. It returns $[u, v]$ only when each count is at least $N/4$ and $u < v$; otherwise it returns `FAIL`. Enclosure then certifies the three probability bounds in Lemma 3.2. Appendix C gives the endpoint selection and proves its success guarantee.

Thus both routines decide by exact finite counts. The same appendix chooses their parameters and shows why a failed test permits a profitable refinement. The next section uses the resulting enumeration budget to obtain a uniform polynomial running time.

7 A uniform polynomial-time implementation

For each fixed degree threshold k, the preceding sections give a finite deterministic adversary. We now bound how long it maintains the recursive state and choose k slowly enough that its local calculations take polynomial time. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

7.1 A finite horizon for the recursive state

Recall the horizon $Q(n) = n \lceil \log n \rceil$ from (1). The algorithm runs the recursive adversary for at most $Q(n)$ answers, recording this prefix of the transcript. If another query arrives, it forms the directed graph of those answers and chooses its lexicographically first topological ordering. It answers that query and every subsequent query from the resulting fixed ranks.

Proposition 4.3 ensures that the prefix is consistent with a total order, so this graph is acyclic and the topological ordering exists. Every later answer remains consistent with the prefix. If the sorter reaches the horizon, it has already made at least $n \log n$ comparisons. If it stops earlier, the comparison bound for the recursive adversary applies.

Lemma 4.6 bounds the number of refinements after q completed calls by $O(q/k)$. An unfinished call adds at most $n + 2$: two while selecting an answer and at most n in the structural repair. Thus the number P of refinements before the horizon, including any call still in progress, satisfies

$$P = O(Q(n)/k + n). \tag{24}$$

The recursive tree has at most $P + 1$ nodes and contains n item records. The stored transcript has at most $Q(n)$ pairs. After switching to fixed ranks, the recursive state is no longer needed.

7.2 An explicit budget for the finite calculations

Appendix C chooses the parameters in the local enclosure theorem. Write S_k for its number of digit choices, t_k for the number of sweeps, and b_k for the bound on retained vertices in (40). The number of digit sequences in one calculation is at most

$$E_k = S_k^{b_k t_k}. \tag{25}$$

For $k = 64, 128, \ldots$, a fixed program writes the complete parameter tuple and E_k in binary. On input size n, simulate this program for n steps. Among the tuples it has completed, choose the largest k with

$$E_k \le n. \tag{26}$$

Use these parameters for the entire recursive phase. If no tuple qualifies, answer all comparisons according to the fixed label order.

The initialization takes polynomial time: the simulation writes at most n bits, and its completed tuples can be parsed and compared within a polynomial bound. For every fixed k, the program finishes its tuple in some finite time $T(k)$. Once $n \ge \max\{T(k), E_k\}$, that tuple is available and meets the budget. Hence the selected $k(n)$ tends to infinity.

7.3 Precision and work per call

Each finite calculation enumerates at most $E_k \le n$ digit assignments. For each assignment, either endpoint calculation uses at most $\log E_k$ updates, so the work involves polynomially many arithmetic operations. A comparison call uses at most three comparison calculations and at most n central-interval calculations, one for each structural refinement.

The numbers also have polynomial length. Let Λ be the greatest number of fractional binary digits in any stored endpoint or anchor. A refinement increases this by at most $\log E_k$, and normalization only copies existing bounds. Hence

$$\Lambda \le P \log E_k = O(n \log^2 n). \tag{27}$$

Appendix E proves this precision bound and accounts for the graph operations, rounding, and exact arithmetic. Together with the finite horizon, these bounds give one fixed polynomial for the work per query, independent of the selected parameters and the length of the interaction. After the horizon, each answer compares two stored ranks.

Theorem 7.1. *There is a deterministic legal adversary, initialized using only n, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

comparisons.

Proof of Theorem 7.1. Initialization and every comparison call are deterministic and take polynomial time. The recursive phase is legal by Proposition 4.3, and the final fixed order respects its recorded answers. For all sufficiently large n, a parameter tuple qualifies. If the sorter reaches $Q(n)$ answers, it has already made at least $n \log n$ comparisons. Otherwise Theorem 4.1 gives

$$q \geq \frac{\log(n!) - n}{1 + O(\log k(n)/k(n))} = (1 - o(1))n \log n,$$

since $k(n) \to \infty$. The fixed-order rule used for the remaining input sizes is deterministic and legal as well. □

8 Consequences and further questions

The counting argument also applies when the algorithm starts or finishes with a partial order. The two settings have different information bounds: initial relations reduce what remains to be learned, whereas a target partial order specifies what the algorithm must establish. We state the consequences here and give their proofs in Appendix G.

8.1 Producing a prescribed partial order

Let R be a partial order on n slots. In *partial-order production*, the input consists of n keys with no known relations. The algorithm must assign one key to each slot so that the key in slot u is smaller than the key in slot v whenever $u <_R v$. Sorting is the case where R is a chain; a slot required to precede every other slot asks for the minimum.

A fixed assignment is valid for exactly $e(R)$ total orders of the keys. The information-theoretic lower bound is therefore

$$I(R) = \log(n!) - \log e(R).$$

Cardinal et al. [14] give a deterministic polynomial-time algorithm using $I(R) + o(I(R)) + O(n)$ comparisons.

Corollary 8.1 (Partial-order production)**.** *There is a nonnegative sequence $\varepsilon_n \to 0$, independent of the target, such that the adversary of Theorem 7.1 forces every correct comparison algorithm producing an n-element partial order R to make at least*

$$(1 - \varepsilon_n)I(R) - \varepsilon_n n$$

comparisons. The adversary is initialized using only n and works for every target R.

The proof counts represented orders after every completed comparison. At a successful production leaf, all these orders must respect the algorithm's chosen assignment, so at most $e(R)$ can remain. A small change to the counting experiment reduces its additive loss from n to $o(n)$; the adversary itself is unchanged.

Thus the adversary forces a $1 - o(1)$ fraction of $I(R)$ whenever $I(R) = \Omega(n)$. Its leading term matches Cardinal et al.'s upper bound when $I(R) = \omega(n)$. For smaller information bounds, the task can require many more comparisons: producing a minimum has $I(R) = \log n$, while finding it requires $n - 1$ comparisons.

8.2 Sorting from supplied partial information

Now let P describe relations already known between the input keys, supplied as a DAG. The sorter must identify one of its $e(P)$ linear extensions, so the information bound is $\log e(P)$.

Corollary 8.2 (Supplied partial information)**.** *There is a deterministic adversary that can be initialized from any n-element partial order P in polynomial time, gives legal answers in polynomial time per comparison, and forces every correct sorter to make at least*

$$(1 - o(1)) \log e(P) - O(n \log \log n)$$

additional comparisons. Both error terms are uniform over P.

The construction uses Cardinal et al.'s polynomial-time procedure [14] to extend P to a *weak order*: a sequence of antichain layers, with every key in an earlier layer preceding every key in a later one. The additional relations reduce $\log e(P)$ by at most $O(n \log \log n)$. Run an independent copy of our adversary within each layer and answer between layers from their fixed order. All copies use the same parameters, chosen from the total size n, so the guarantee also covers small layers.

The comparison bound approaches $\log e(P)$ when $\log e(P) = \omega(n \log \log n)$. Kahn and Kim's adversary already forces $\Omega(\log e(P))$ for every P; the improvement here is the coefficient approaching one in this regime. Appendix G.4 also gives a direct initialization using partial-order production, with an additive $o(n \log n)$ loss.

8.3 Preserving small amounts of remaining uncertainty

Can a deterministic polynomial-time adversary force $(1 - o(1)) \log e(P)$ further comparisons for every supplied partial order P? The additive loss in Corollary 8.2 can overwhelm the information that remains. Two disjoint chains of length $n/2$ have $\log e(P) = \Theta(n)$; a chain on $n - 1$ items and one unrelated item have $e(P) = n$.

Reducing the additive loss to $O(n)$ would already extend the useful range. Cardinal et al. [14, Section 4] exhibit partial orders P for which every weak-order extension W satisfies $\log e(W) \le \log e(P) - \Omega(n \log \log n)$. A sharper construction must therefore preserve more of the possible interleavings between layers.

Initial partial information also arises when sorting points whose locations are known only to lie in specified intervals [18, 19]. Extending our adversary to those models would require answers compatible with realizable coordinates and with the permitted geometric operations. Another direction is the amortized minimum-maintenance problem linked to sorting adversaries by Kaplan, Zamir, and Zwick [7, Section 7]: handling insertions and deletions would require controlling how they change the represented orders.

A Canonical intervals and densities

We first verify that normalization stores the full range of each coordinate without changing the feasible region. We then establish the density bounds used to choose anchors and analyse probability certificates.

Lemma A.1 (Canonical intervals and sections)**.** *Algorithm 4 preserves the feasible region and its volume. After normalization, the region has positive volume if and only if all stored widths $r_x - \ell_x$ are positive. In that case (ℓ_x, r_x) is the projection of the region onto coordinate x, and fixing any $X_x = c \in (\ell_x, r_x)$ leaves a positive-volume section.*

Proof. Every propagated bound is already implied by an arc and an interval bound. A forward topological pass propagates every lower bound to all its successors; the reverse pass propagates every upper bound to all its predecessors. Thus normalization preserves the region and gives $\ell_x \le \ell_y$ and $r_x \le r_y$ along every arc $x \to y$.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Along an arc $x \to y$, increasing θ_x to θ_y strictly increases this value, because $r_x > \ell_x$. Replacing the endpoints by ℓ_y, r_y cannot decrease it. Thus $X_x < X_y$, all interval bounds hold strictly, and a small open neighborhood is feasible.

Now fix $X_x = c$ in the stored interval and delete its coordinate. Incident arcs impose the upper bound c at its in-neighbors and the lower bound c at its out-neighbors. Lower bounds propagate forward and upper bounds backward. Any earlier lower bound that reached a successor through x is superseded by $c > \ell_x$; any earlier upper bound that reached a predecessor through x is superseded by $c < r_x$. The normalized intervals of the remaining coordinates are therefore

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases} \tag{28}$$

A predecessor retains positive width because $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too because $r_y > \ell_y$ and $c < r_x \leq r_y$. Other widths are unchanged. The criterion just proved shows that the section has positive volume. Since this holds for every $c \in (\ell_x, r_x)$, that interval is exactly the coordinate projection. □

A.1 Bounds from convex sections

Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \neq x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \operatorname{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\operatorname{vol}_N K}.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.2 (Density of a linear projection)**.** *Let $C \subseteq \mathbb{R}^d$ be a bounded open convex region of positive volume, and let φ be a nonzero linear functional with $\varphi(C) = (\ell, r)$ of width W. For X uniform on C, the density g of $\varphi(X)$ satisfies*

$$g(t) \leq \frac{d}{W}, \qquad g(t) \geq \frac{4^{-(d-1)}}{W} \quad (\ell + W/4 \leq t \leq r - W/4).$$

Proof. An invertible linear change of coordinates makes φ the first coordinate. Its constant Jacobian preserves uniformity on the transformed region, so it suffices to prove the bounds for a coordinate projection. Write $S(t)$ for the section volume at that coordinate's value t; then $g(t) = S(t)/\operatorname{vol} C$.

Choose points in $\overline{C}$ whose first coordinates are ℓ and r. Such points exist because C is bounded. For $\ell < t < s < r$, join every point of the section at t to the point at r. Since C is open and

convex, the image with first coordinate s lies in the section of C there. Each of its $d-1$ directions is scaled by $(r-s)/(r-t)$, so

$$S(s) \geq \left(\frac{r-s}{r-t}\right)^{d-1} S(t) \qquad (t < s). \tag{29}$$

Joining instead to the point at ℓ gives

$$S(s) \geq \left(\frac{s-\ell}{t-\ell}\right)^{d-1} S(t) \qquad (s < t). \tag{30}$$

Normalize these inequalities and integrate over s on both sides of t:

$$1 \geq g(t)\left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^{d-1} ds + \int_t^r \left(\frac{r-s}{r-t}\right)^{d-1} ds\right) = \frac{W g(t)}{d}.$$

Thus $g(t) \leq d/W$.

For the lower bound, choose t_0 with $g(t_0) \geq 1/W$, which exists because g integrates to one over an interval of length W. For any $s \in [\ell + W/4, r - W/4]$, the appropriate ratio in (29) or (30) is at least $1/4$. Consequently

$$g(s) \geq 4^{-(d-1)} g(t_0) \geq \frac{4^{-(d-1)}}{W}.$$

The case $s = t_0$ satisfies the same bound directly. □

Lemma A.3 (Coordinate density bounds)**.** *Suppose a node's feasible region has positive volume, and x has at most B other comparable coordinates and canonical interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \leq \frac{B+1}{W}, \tag{31}$$

$$f_x(t) \geq \frac{4^{-B}}{W} \qquad (\ell + W/4 \leq t \leq r - W/4). \tag{32}$$

Proof. Fix all coordinates incomparable with x, at values leaving a nonempty region. By the section formula in Lemma A.1, each such fixing leaves x's canonical interval unchanged. The remaining region C is open and convex, has dimension $d \leq B+1$, and still projects onto (ℓ, r) in coordinate x. Lemma A.2 therefore gives both required bounds for the density of x in C.

Average over the fixed coordinates. If ω denotes their values, C_ω the remaining region, and g_ω its coordinate density, integration of each section gives the pointwise identity

$$f_x(t) = \frac{\int \mathrm{vol}(C_\omega) g_\omega(t)\, d\omega}{\int \mathrm{vol}(C_\omega)\, d\omega}.$$

Empty slices contribute zero. The bounds on g_ω are uniform in ω, so they also hold for f_x. □

Lemma A.4 (A bound for small differences)**.** *Let X be uniform on a positive-volume node region, and suppose x has at most B other comparable coordinates and canonical interval of width W. Let Y be a fixed number or X_y for any distinct free coordinate y. For every interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \leq \frac{(B+1)a}{W}. \tag{33}$$

Proof. For a fixed number Y, integrate the coordinate-density upper bound over $I + Y$. Now let $Y = X_y$ for any free coordinate $y \neq x$. As above, fix all coordinates incomparable with x. The remaining convex region C has dimension $d \leq B + 1$, and x retains its width W. We show that the projection $X_x - X_y$ has width at least W in each such region.

If y is incomparable with x, it is now fixed, so the projection is a translate of X_x and has width W. Otherwise y remains free. Write (ℓ_z, r_z) for canonical intervals in C; x's endpoints are unchanged. If a directed path goes from x to y, then $r_y \geq r_x$. Fixing $X_y = t \in (\ell_y, r_y)$ leaves x the interval $(\ell_x, \min(r_x, t))$, by Lemma A.1. As t approaches r_y, its width tends to W. On that section the difference $X_x - t$ varies over an interval of the same width. Thus the projection of the whole region C has width at least W. If a directed path goes from y to x, let t approach ℓ_y instead. Here $\ell_y \leq \ell_x$, and the remaining interval $(\max(\ell_x, t), r_x)$ again has width tending to W.

Lemma A.2 bounds the conditional density of the difference by $d/W \leq (B+1)/W$. Integrating over I and averaging over the fixed coordinates proves (33). If both coordinates have at most B comparable coordinates, exchanging x and y gives the same bound with $\max(W_x, W_y)$ in the denominator. □

A.2 Density in a central interval

A density is *log-concave* if its positive support is convex and its logarithm is concave there. The coordinate densities above are log-concave pointwise: the uniform density on K is log-concave, and integrating out the other coordinates preserves this property by Prékopa's marginal theorem [13, Theorem 8]. We finish with the density bound used for positions having substantial probability on each side.

Lemma A.5 (Density at central positions)**.** *Let X have a log-concave density f. If $\mathbb{P}(X \leq c) \geq p$ and $\mathbb{P}(X \geq c) \geq p$ for some $0 < p \leq 1/2$, then*

$$f(c) \geq p \sup_t f(t).$$

Proof. For $s \leq c \leq t$, log-concavity gives

$$f(s)f(t) \leq f(c)f(s + t - c).$$

Indeed, c and $s + t - c$ are complementary convex combinations of s, t; multiply the two log-concavity inequalities. Fixing $t \geq c$ and integrating over $s \leq c$ gives

$$pf(t) \leq f(c) \int_{-\infty}^{c} f(s + t - c)\, ds \leq f(c).$$

Here the first inequality uses the mass at least p to the left of c, and the last uses that a density integrates to one. Fixing $s \leq c$ and integrating over $t \geq c$ instead uses the right-hand mass and gives $pf(s) \leq f(c)$. Taking the supremum over both sides of c proves the claim. □

Proof of Lemma 5.2. Put $M = \sup_t f_x(t)$. The inside-mass condition gives $M|J| \geq 1/4$. Every $c \in J$ has probability at least $1/4$ on each side, so Lemma A.5 gives $f_x(c) \geq M/4 \geq 1/(16|J|)$. □

B Why the enclosing intervals become narrow

The enclosure part of Theorem 6.1 was proved in Section 6. Here we bound the fraction of digit sequences that leave a wide interval at a queried coordinate. We compare the lower and upper calculations with each other. Their initial widths are at most one, and their boundary values stay fixed. For the analysis, choose each interior digit independently and uniformly; each enumerated sequence then has probability $1/N$.

B.1 The decrease after one sweep

Write $\Delta_z = B_z - A_z$. Superscripts old and new refer to the start of a sweep and the value after a coordinate's update. For an interior vertex z, the shared digit j_z gives

$$\Delta_z^{\text{new}} \le \frac{1}{S}\Delta_z^{\text{old}} + \frac{S-1-j_z}{S}\max_{u\to z}\Delta_u^{\text{new}} + \frac{j_z}{S}\max_{z\to u}\Delta_u^{\text{old}}, \tag{34}$$

with an empty maximum equal to zero. The stored interval bounds in the two calculations are identical.

Let $B_h(z)$ be the vertices at undirected distance at most h from z, and suppose they are all interior vertices. Put $D = \max_{u\in B_h(z)}\Delta_u^{\text{old}}$ and $a_u = (S-1-j_u)/S$. At z, (34) gives

$$\Delta_z^{\text{new}} \le (1-a_z)D + a_z\max_{u\to z}\Delta_u^{\text{new}}.$$

Follow a predecessor attaining the maximum, and continue backwards to a source. This path has at most h vertices; all its vertices and their successors lie in $B_h(z)$. At the source the predecessor term is zero. Substitution along the path bounds the new width by $\left(1-\prod_u a_u\right)D$.

Write $\Delta_z(s)$ for the width after s sweeps. Call the pair (z,s) *bad* if, during sweep s, the digit at z or any vertex preceding z on a directed path equals $S-1$. Otherwise all factors a_u in the preceding expansion are at least $1/S$, and there are at most h of them. Put $\beta = 1-S^{-h}$. Consequently,

$$\Delta_z(s) \le \kappa_z(s)\max_{u\in B_h(z)}\Delta_u(s-1), \qquad \kappa_z(s) = \begin{cases} 1, & (z,s) \text{ is bad,} \\ \beta, & \text{otherwise.} \end{cases} \tag{35}$$

B.2 Averaging powers of the widths

We must control a maximum over several earlier widths. Taking a power and bounding the maximum by a sum makes this possible when the expected contraction is strong enough. The following lemma also allows the set of vertices under consideration to shrink after each sweep.

Lemma B.1 (Averaging contraction)**.** *Let $V_0 \supseteq \cdots \supseteq V_t$, and let nonnegative quantities $D_z(s)$ be determined by the first s sweeps. Suppose $D_z(0) \le 1$ and, for $z \in V_s$, there are nonnegative multipliers $\kappa_z(s)$ with*

$$D_z(s) \le \kappa_z(s)\max_{u\in N(z)} D_u(s-1), \qquad N(z)\subseteq V_{s-1}, \quad 1 \le |N(z)| \le M.$$

If, for an integer $p \ge 1$, the conditional expectation of $\kappa_z(s)^p$ given all earlier sweeps is at most $1/(2M)$, then for $z \in V_s$ and every $\delta > 0$,

$$\mathbb{E}D_z(s)^p \le 2^{-s}, \qquad \mathbb{P}(D_z(s) > \delta) \le 2^{-s}\delta^{-p}. \tag{36}$$

Proof. The old widths are fixed when conditioning on the earlier sweeps. Since the maximum of their p-th powers is at most their sum,

$$\mathbb{E}D_z(s)^p \le \frac{1}{2M}\sum_{u\in N(z)}\mathbb{E}D_u(s-1)^p \le \frac{1}{2}\max_{u\in N(z)}\mathbb{E}D_u(s-1)^p.$$

Induction gives the moment bound. The probability bound follows by Markov's inequality. □

B.3 Choosing the parameters

Write $r = 2$ for the bound on the number of queried coordinates. Both $B_h(z)$ and the set consisting of z and its directed-path predecessors have at most M vertices. Choose

$$M = (d+1)^{h+1}, \qquad S = \text{the least power of two at least } 4M^2. \tag{37}$$

The bad event in (35) involves only fresh digits from the current sweep. Its conditional probability is at most $M/S \le 1/(4M)$. With $\beta = 1 - S^{-h}$, choose the smallest positive integer p satisfying the rational test

$$\beta^p \le \frac{1}{4M}. \tag{38}$$

Consequently,

$$\mathbb{E}[\kappa_z(s)^p \mid \text{earlier sweeps}] \le \frac{1}{4M} + \beta^p \le \frac{1}{2M}.$$

This estimate uses all directed-path predecessors in defining the bad event, so it also covers the predecessor choices made in the one-sweep expansion. No independence between vertices within a sweep is required.

Choose the smallest positive integer t satisfying

$$2^{-t} \le \frac{\alpha\delta^p}{r}, \qquad \rho = ht + 1. \tag{39}$$

These choices are computable: $0 < \beta < 1$, and both tests compare rational numbers. Retain radius ρ around the queried set T, freezing the outer boundary as in Algorithm 6.

For $0 \le s \le t$, let V_s consist of the vertices at distance at most $h(t-s)$ from T. If $z \in V_s$ and $s \ge 1$, then $B_h(z) \subseteq V_{s-1}$. Every vertex in these sets has distance at most $ht < \rho$ from T, so all the updates used by the one-sweep inequality occur in the interior. Lemma B.1 therefore applies to $D_z(s) = \Delta_z(s)$ and $N(z) = B_h(z)$. At the queried vertices, which form $V_t = T$, it gives

$$\mathbb{P}(\Delta_z(t) > \delta) \le 2^{-t}\delta^{-p} \le \alpha/r.$$

A union bound over at most r queried coordinates proves the width guarantee of Theorem 6.1.

Each radius-ρ neighborhood has at most $1 + d + \cdots + d^\rho \le (d+1)^{\rho+1}$ vertices. We may therefore use

$$b = r(d+1)^{\rho+1}, \qquad E = S^{bt} \tag{40}$$

as bounds on the retained vertices and enumerated digit assignments. All these choices depend only on d, h, δ, α.

C Calibration of the finite certificates

We verify the central-interval routine and choose the finite calculation's parameters. The purpose of the calibration is to ensure that a failed certificate identifies a coordinate whose short interval makes a middle-half refinement profitable.

C.1 The central-interval test

For the one-coordinate list $[A_i, B_i]$, $1 \le i \le N$, take u to be the $N/4$-th upper endpoint in sorted order and v to be the $3N/4$-th lower endpoint. The alphabet size S is divisible by four, and the queried coordinate is interior, so $N/4$ is an integer. Return `FAIL` if $u \ge v$ or fewer than $N/4$ list intervals lie wholly in $[u, v]$. Otherwise return $[u, v]$.

Proof of Lemma 3.2. At least $N/4$ intervals end at or below u, and at least $N/4$ begin at or above v. On success, at least $N/4$ intervals also lie wholly inside $[u, v]$. Enclosure certifies mass at least $1/4$ in each of the three regions. The true distribution is continuous and supported on (ℓ_x, r_x), so these masses imply $\ell_x < u < v < r_x$. The endpoints are dyadic, and sorting and counting finite lists are deterministic. □

C.2 The width at which certificates succeed

The graph bounds limit how many coordinates can be comparable with a queried coordinate. Define

$$B = (k+1)^{2H+1}. \tag{41}$$

In an in-node DAG of degree at most k and directed paths of at most $2H$ vertices, every comparable coordinate lies within undirected distance $2H$. Thus B bounds the number of other comparable coordinates. Set

$$w_* = 2^{-2kB}. \tag{42}$$

This width calibrates the finite calculations. The algorithms test the computed certificates directly.

Lemma C.1 (When certification succeeds)**.** *Suppose a node has positive feasible volume, its in-node DAG has degree at most k, and its directed paths have at most $2H$ vertices. If any free coordinate involved in a comparison has interval width at least w_*, comparison certification succeeds. If $W_x \geq w_*$, then* `CentralInterval`(v, x) *succeeds.*

Proof of Lemma C.1. Calibrate both routines using Theorem 6.1 with

$$d = k, \qquad h = 2H, \qquad \alpha = \frac{\eta}{2}, \qquad \delta = \frac{\eta w_*}{8(B+1)}. \tag{43}$$

These parameters are fixed for the interaction. They do not depend on the queried widths.

Suppose one queried free coordinate x has $W_x \geq w_*$. If the computed intervals overlap and both have width at most δ, the enclosed positions differ by at most 2δ. Lemma A.4 therefore bounds the unresolved fraction by

$$1 - p - q \leq \alpha + \mathbb{P}(|X_x - X_y| \leq 2\delta) \leq \alpha + \frac{4(B+1)\delta}{w_*} = \eta.$$

Thus $\max(p, q) \geq (1-\eta)/2 > 1/2 - \eta$, so comparison certification succeeds. For a threshold c, an unresolved interval contains c. If its width is at most δ, its enclosed position is within δ of c, giving the stronger bound

$$\alpha + \frac{2(B+1)\delta}{w_*} = \frac{3\eta}{4}. \tag{44}$$

It remains to show that `CentralInterval` succeeds when $W_x \geq w_*$. By (44), at most a fraction $\tau = 3\eta/4$ of its intervals contain any fixed real number. This bound applies in particular to the selected endpoints u and v. An interval extending below u either ends strictly below u or contains u. By the rank defining u, fewer than $N/4$ intervals end strictly below it. Thus at most $(1/4 + \tau)N$ intervals extend below u. Similarly, at most $N/4$ intervals begin strictly above v, so at most $(1/4 + \tau)N$ extend above v.

At least $(1/2 - 2\tau)N \geq N/4$ intervals therefore lie wholly inside $[u, v]$, since $\eta < 1/32$. Enclosure gives this interval mass at least $1/4$. Continuity forces $u < v$, so both success tests pass. □

D Refinement and consistency

D.1 The refinement cost guarantee

We prove Lemma 4.5. The successful central choice was analysed in (21); the remaining case chooses an anchor in the middle half of the coordinate's interval.

Proof of Lemma 4.5. The in-node DAG bounds ensure that the number of other coordinates comparable with x is at most B from (41). Suppose the routine uses the middle-half interval. Either comparison certification failed, or a structural refinement's call to `CentralInterval` failed. In both cases, Lemma C.1 gives $W_x < w_* = 2^{-2kB}$. Write $L = \log(1/W_x) > 2kB$. The coordinate-density bound (32) and the refinement identity give

$$g = \log f_x(c) \geq L - 2B.$$

Since the rounded interval has length $W_x/2$, Lemma 5.1 gives $m < L + 1$. Consequently

$$g \geq L - 2B > (1 - 1/k)L > (1 - 1/k)m - 4.$$

The central choice satisfies the same final bound, by (21). Thus in every case

$$g \geq (1 - 1/k)m - 4. \tag{45}$$

In particular $g + 4 \geq 0$. If the refinement follows failed comparison certification, it uses the middle half directly and has

$$g \geq L - 2B > 2B(k - 1) > k. \tag{46}$$

Both choices return an interior dyadic anchor and terminate.

Combining (45) with the child cost (14) gives

$$\ell(c) \leq \frac{\gamma}{1 - 1/k}(g + 4) + \log k + 1.$$

For $k \geq 64$, the estimate $-\log(1 - 1/k) \leq 1/((k - 1)\ln 2) \leq 3/(2k)$ gives $\gamma/(1 - 1/k) \leq 1 + 3/k$. Set

$$a = 1 + \frac{3}{k}, \qquad b = 4a + \log k + 1.$$

Because $g + 4 \geq 0$, increasing its coefficient to a preserves the inequality. Thus $\ell(c) \leq ag + b$, including when g is negative. The coefficients have $a = 1 + O(1/k)$ and $b = O(\log k)$, with $a \geq 1$ and $1 \leq b \leq \log k + 6 < k + 1$. For $k \geq 64$, they also satisfy $a \leq 3/2$ and $b \leq k/4$, since $\log k + 6 \leq k/4$. □

D.2 Consistency and termination

We first justify admission to an existing child, then verify consistency through an entire comparison call. Assignments with equal free coordinates, or with a free coordinate equal to an anchor, form a finite union of sets of volume zero. They are excluded when reading a total order from the state.

Proof of Lemma 5.3. Consider first an existing child at c, and denote its label set by S. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma A.1. Also choose feasible assignments in the other nodes.

There is a positive distance from c to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at x are strict. Thus, with everything else fixed, x

can be moved slightly to either side of c while keeping the old state feasible. The resulting two represented orders place x immediately before or immediately after the entire block S. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the transcript.

Now take any order represented after the refinement. Deleting x from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except x. The new order inserts x somewhere into their child block. No answered comparison joins x to a label in S, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item x is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at c, the same argument uses an empty block: the new singleton child places x where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

Proof of Proposition 4.3. Initially every node has positive volume, every order is represented, and the strict in-node DAG bounds hold. We induct over completed calls and, within each call, over its changes to the state.

Before answering, a refinement deletes one vertex and adds an isolated vertex to a child, so the strict DAG bounds continue to hold. The anchor routine returns an interior position, as proved in Appendix D.1. Lemma 5.3 preserves positive node volumes and consistency with the transcript. Normalization preserves the represented region by Lemma A.1.

If the queried labels lie in different child subtrees, their anchors determine the answer in every represented order. Otherwise, the acceptance test in Algorithm 1 and Lemma 3.1 ensure that the chosen answer retains at least $1/2 - \eta > 0$ of its node's volume. Recording its arc or interval bound therefore preserves positive volume and makes the answer true in every represented order. A new arc cannot create a cycle, since that would make the region empty. Normalization preserves this updated region. The transcript is then extended by the chosen answer.

During the subsequent repair, the temporary DAG bounds follow from Lemma 5.4. Each move again preserves positive volume and the transcript by Lemma 5.3. This applies to every vertex of a selected path, including those whose incident arcs were removed earlier in the batch. The full batch is completed, and Lemma 5.4 restores the strict bounds for the next call.

The finite certificate calculations and the explicit rounding rule terminate. As shown in Section 5.2, before answering each queried label can be refined only once: a second move would have gain zero, contradicting the failed-certification gain guarantee. Thus there are at most two such moves. A repair makes at most n further moves, so every call terminates with at most $n + 2$ refinements. All analytic guarantees used here apply to arbitrary positive-volume regions with the stated DAG bounds, independently of the interaction history. □

E Exact arithmetic and the stored state

The budget $E_k \leq n$ bounds the number of digit assignments and their length. We verify that it also bounds their bit cost, and that the recursive state remains of polynomial size up to the comparison horizon.

E.1 Precision in the interval calculation

Suppose the stored endpoints have at most Λ fractional binary digits. An update takes maxima and minima, forms an integer linear combination, and divides by the power of two S. It therefore adds at most $\log S$ fractional digits. If there are b' interior vertices, each endpoint calculation makes $b't$ updates. The two calculations share digits but use their own values, so the precision in either is at most

$$\Lambda + b't \log S \le \Lambda + \log E. \tag{47}$$

This proves the precision claim of Theorem 6.1. All computed positions remain in $[0, 1]$, so their integer numerators need at most one additional bit. Digits and counts have at most $\log S$ and $\log E + 1$ bits, respectively.

E.2 Precision of new anchors

If a nondegenerate interval has dyadic endpoints with at most F fractional digits, its length is at least 2^{-F}. The rounding rule of Lemma 5.1 therefore uses a grid exponent at most F, and its anchor has at most F fractional digits.

For a certified central interval, (47) gives $F \le \Lambda + \log E_k$. Forming the middle half of a stored interval gives endpoints with at most $\Lambda + 2$ digits. Since $E_k \ge 4$, both choices produce anchors with at most $\Lambda + \log E_k$ digits. This also covers the direct middle-half choice after a failed comparison certificate. The update of neighboring bounds and `Normalize` take maxima and minima of existing endpoints and the chosen anchor, adding no precision.

All initial endpoints are 0 or 1. After P refinements, induction therefore gives $\Lambda \le P \log E_k$, as claimed in (27). The bound includes the stored state during an unfinished call. Temporary endpoints in the next calculation add at most another $\log E_k$ digits.

E.3 Inventory and work per call

Each refinement creates at most one recursive-tree node. There are thus at most $P + 1$ nodes, n item records, and at most P stored child anchors. An item belongs to exactly one in-node DAG, so their total numbers of vertices and arcs are at most n and n^2, respectively. The recorded prefix contains at most $Q(n)$ comparisons. Labels, pointers, depths, and counters have polynomial length, as do all dyadic endpoints by (27).

For one certificate, the algorithm enumerates at most $E_k \le n$ digit assignments. Each assignment uses two endpoint calculations, each with at most $b_k t_k \le \log E_k \le \log n$ updates. The boundary vertices remain fixed. Maxima, minima, integer multiplication, and division by powers of two operate on polynomially many bits. Central certification sorts at most n lower and upper endpoints and counts the intervals inside its candidate. Rounding computes a grid exponent and a ceiling using the same bounded-precision integers. Standard binary arithmetic therefore gives a fixed polynomial bound for each operation.

Before selecting an answer, a call makes at most two failure refinements and three calls to `Certify`. Failure refinements use the middle half directly. After recording the answer, the single structural repair makes at most n refinements and hence at most n calls to `CentralInterval`.

The remaining work is polynomial in the stored graph size. Topological passes normalize intervals and compute directed-path lengths. A finite graph search finds each retained neighborhood; it stops when either the prescribed radius or all available vertices have been reached. Routing uses the recursive tree, and deterministic choices can be made by scanning labels. At the horizon, topologically sorting the bounded transcript DAG produces the fixed ranks for all later answers. Thus the bit-operation bound has a fixed exponent, independent of k and of the number of queries already answered.

F Parameter reference

Table 1 collects the parameters of the adversary and its analysis. The last two columns distinguish where a parameter's role is introduced from where its value is chosen and explained. All choices are fixed before the first query. Appendix B gives explicit choices for the digit alphabet, number of sweeps, and neighborhood radius in the local enclosure theorem.

Table 1: Parameters and their roles. References lead to the explanations in the text.

Symbol	Value	Role	Introduced	Choice
k	Integer, $k \geq 64$	Repair a vertex that reaches this degree after an answer.	§3.2	§4
H	2^k	Number of vertices refined in a selected directed path.	§3.2	§4.3
B	$(k+1)^{2H+1}$	Bounds the number of other vertices comparable with one vertex in its in-node DAG.	App. C	App. C
w_*	2^{-2kB}	Calibrates the finite calculations: a coordinate of at least this width guarantees certification.	App. C	App. C
η	$1/(100k)$	Accept an answer whose certified probability is at least $1/2 - \eta$.	§3.2	§4.1
$Q(n)$	$n \lceil \log n \rceil$	Number of answers before switching to a fixed consistent order.	§3.2	§3.2

G Proofs of the consequences

We first strengthen the count of orders remaining after any comparison prefix. This gives the production corollary. We then use an ordered partition supplied by Cardinal et al. to handle initial partial information, and give an alternative initialization through production.

G.1 Orders remaining after a comparison prefix

Rescaling the weights used in the counting experiment of Section 4.2 leaves the adversary and all refinement estimates unchanged. Reducing the chance of entering a child increases the chance of stopping at the required storage node. This reduces the loss incurred once per item, at an additional cost each time an item moves. Lemma 4.6 bounds how often that cost is incurred.

Lemma G.1 (Orders remaining after any prefix)**.** *For every sufficiently large fixed k, put $d_k = -\log(1 - 1/k) = O(1/k)$. There is a nonnegative $\delta_k = O(\log k/k)$ such that, after any q completed calls in the recursive phase, the number N_{ord} of represented orders satisfies*

$$\log N_{\mathrm{ord}} \geq \log(n!) - nd_k - (1 + \delta_k)q. \tag{48}$$

The constants are independent of n and of the comparison sequence.

Proof. Use the same experiment as in Lemma 4.4, with child weights

$$\widetilde{w}(c) = \frac{2}{k} w(c).$$

The total child weight is at most $1/k$, so the stopping probability at each node is at least $1 - 1/k$. Let M be the number of refinements, and let $\widetilde{\mathcal{W}}$ be the product of these new weights along all item paths. The argument of Lemma 4.4 gives

$$N_{\mathrm{ord}} \geq n!(1-1/k)^n Z\widetilde{\mathcal{W}}, \qquad -\log \widetilde{\mathcal{W}} = \sum_r \ell(c_r) + M\log(k/2).$$

Summing the unchanged refinement estimate (15) and using (17) yields

$$\sum_r \ell(c_r) \leq a\log Z + aa_\eta q + bM.$$

Consequently

$$\log N_{\mathrm{ord}} \geq \log(n!) - nd_k + (1-a)\log Z - aa_\eta q - \big(b + \log(k/2)\big)M.$$

Here $(1-a)\log Z \geq 0$, $aa_\eta = 1 + O(1/k)$, $b = O(\log k)$, and $M = O(q/k)$ by Lemma 4.6. These bounds prove (48) with a nonnegative $\delta_k = O(\log k/k)$. □

G.2 Producing a target order

Proof of Corollary 8.1. For sufficiently large n, let $k = k(n)$ be the parameter selected in Section 7.2. If the production algorithm reaches the horizon $Q(n)$, then it has made at least $n\log n \geq I(R)$ comparisons. Otherwise it finishes after $q < Q(n)$ completed calls. Its output assigns each key to one slot of R. Correctness requires that every total order consistent with its comparison transcript respect this assignment. Exactly $e(R)$ total orders respect the assignment, and all represented orders are consistent with the transcript. Thus $N_{\mathrm{ord}} \leq e(R)$, and Lemma G.1 gives

$$q \geq \frac{I(R) - nd_{k(n)}}{1 + \delta_{k(n)}}. \tag{49}$$

Both $d_{k(n)}$ and $\delta_{k(n)}$ tend to zero. Taking $\varepsilon_n = \max\{d_{k(n)}, \delta_{k(n)}\}$ and using $(1+\delta)^{-1} \geq 1 - \delta$ gives the stated bound. Set $\varepsilon_n = 1$ for the finitely many smaller input sizes, making their bound nonpositive. These choices are independent of R. The adversary's legality and running time are those of Theorem 7.1. □

G.3 A compatible partition into layers

The reduction for an initial partial order needs an ordered partition whose layers retain many permutations. We use the following consequence of Cardinal et al.'s construction.

Lemma G.2 (Weak-order extension)**.** *Given a partial order P on n elements, one can deterministically construct in polynomial time a weak order W extending P such that*

$$\log e(W) \geq \log e(P) - C_0 n\log\log n$$

for an absolute constant C_0.

Proof. Cardinal et al. [14, Section 3, Lemmas 1 and 3] construct W in polynomial time with

$$I(W) \leq I(P) + 2n\log\big(I(P)/n + 1\big) + O(n).$$

Since $I(P) \leq \log(n!) \leq n\log n$, the extra term is $O(n\log\log n)$. Substitute $I(W) - I(P) = \log e(P) - \log e(W)$. □

An arbitrary partition into antichains need not admit a compatible layer order. For example, the relations $a < b$ and $c < d$ permit the antichains $\{a, d\}$ and $\{b, c\}$, but require opposite orders between them. Lemma G.2 supplies both the partition and a layer order respecting all initial relations.

Proof of Corollary 8.2. Construct W using Lemma G.2, and let its layers have sizes $m_1, \dots, m_t$. Choose $k = k(n)$ once, using the full input size. Within every layer of size at least two, run a copy of the ordinary adversary with these parameters and its local horizon $Q(m_i) = m_i \lceil \log m_i \rceil$. A singleton needs no comparisons. Answer comparisons between layers from their fixed order.

Each local transcript has a consistent total order. Concatenating these orders gives a linear extension of W and hence of P, proving legality. At termination, each layer must have a unique possible order: two possible orders in one layer, combined with the same orders in the others, would give two global orders consistent with the full transcript.

Let q_i count comparisons within layer i. If its horizon is reached, $q_i \geq m_i \log m_i \geq \log(m_i!)$. Otherwise Lemma G.1, with $N_{\mathrm{ord}} = 1$, gives

$$q_i \geq \frac{\log(m_i!) - m_i d_k}{1 + \delta_k}.$$

This inequality also holds for a singleton and for a layer whose horizon was reached. Since $\sum_i m_i = n$ and $e(W) = \prod_i m_i!$, summing gives

$$\begin{aligned} q \geq \sum_i q_i &\geq \frac{\log e(W) - n d_k}{1 + \delta_k} \\ &\geq \frac{\log e(P) - C_0 n \log\log n - n d_k}{1 + \delta_k}. \end{aligned}$$

As $k(n) \to \infty$, this implies the stated bound, uniformly over P.

The common choice of $k(n)$ is essential to this estimate: each layer has the same d_k and δ_k, even when its size stays small. It also preserves polynomial time in n. Each finite calculation enumerates at most $E_k \leq n$ assignments. Across all layers, the horizons sum to at most $n \lceil \log n \rceil$, and the refinement and precision bounds of Section 7 apply with the ambient size n. The construction of W is polynomial-time as well. If the parameter selection produces no eligible tuple, use a fixed topological order of P; enlarging the absolute error constant covers these finitely many input sizes. □

G.4 Initialization by partial-order production

There is also a direct way to initialize from P, using a production algorithm as a black box. It gives the bound

$$q \geq \log e(P) - r(n), \qquad r(n) = o(n \log n),$$

with one function r for all n-element initial orders.

Run Cardinal et al.'s deterministic production algorithm on target P, answering its comparisons with our ordinary adversary on temporary keys. Its comparison phase uses the deterministic multiple-selection algorithm of Kaligosi et al. [20]. The producer returns an assignment of those keys to the elements of P. Save the adversary's state. Translate each subsequent query through this assignment and continue the same adversary. Every order consistent with the initialization transcript respects the returned assignment, so all later answers are legal for P.

The producer followed by any correct sorter for P is a correct ordinary sorting algorithm. Write the ordinary adversary's guarantee as $\log(n!) - s(n)$, where $s(n) = o(n \log n)$. The production upper bound [14, Theorem 4] gives $U(P) \leq I(P) + u(n)$ with $u(n) = o(n \log n)$ independent of

P: for each fixed $\delta > 0$ its bound is at most $(1+\delta)I(P) + C_\delta n$, and $I(P) \le n \log n$. Subtracting the initialization comparisons gives

$$q \ge \log(n!) - s(n) - U(P) \ge \log e(P) - s(n) - u(n).$$

Both the initialization and every subsequent response take polynomial time. This construction needs no computation of $e(P)$.

References

[1] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching.* Second edition, Addison-Wesley, 1998. Publisher's page.

[2] Michael L. Fredman. *How good is the information theory bound in sorting?* Theoretical Computer Science 1(4), 355–361, 1976. doi:10.1016/0304-3975(76)90078-5.

[3] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[4] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[5] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[6] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[7] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[8] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[9] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[10] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[11] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[12] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.

[13] András Prékopa. *On logarithmic concave measures and functions.* Acta Scientiarum Mathematicarum 34, 335–343, 1973. Publisher's paper.

[14] Jean Cardinal, Samuel Fiorini, Gwenaël Joret, Raphaël M. Jungers, and J. Ian Munro. *An efficient algorithm for partial order production.* SIAM Journal on Computing 39(7), 2927–2940, 2010. doi:10.1137/090759860. Authors' manuscript.

[15] Bernhard Haeupler, Richard Hladík, John Iacono, Václav Rozhoň, Robert E. Tarjan, and Jakub Tětek. *Fast and simple sorting using partial information.* Algorithmica 88, article 41, 2026. doi:10.1007/s00453-026-01387-3.

[16] Ivor van der Hoog and Daniel Rutschmann. *Tight bounds for sorting under partial information.* Proceedings of the 65th IEEE Annual Symposium on Foundations of Computer Science (FOCS), 2243–2252, 2024. doi:10.1109/FOCS61266.2024.00131.

[17] Daniel Rutschmann. *Sorting under partial information with optimal preprocessing time via unified bound heaps.* arXiv:2604.12653, 2026. Author's manuscript.

[18] Ivor van der Hoog, Irina Kostitsyna, Maarten Löffler, and Bettina Speckmann. *Preprocessing ambiguous imprecise points.* 35th International Symposium on Computational Geometry (SoCG), LIPIcs 129, 42:1–42:16, 2019. doi:10.4230/LIPIcs.SoCG.2019.42.

[19] Ivor van der Hoog, Irina Kostitsyna, Maarten Löffler, and Bettina Speckmann. *Sorting under partial (interval order) information.* Journal of Computational Geometry 15(1), 143–171, 2024. doi:10.20382/jocg.v15i1a6.

[20] Kanela Kaligosi, Kurt Mehlhorn, J. Ian Munro, and Peter Sanders. *Towards optimal multiple selection.* Automata, Languages and Programming (ICALP), LNCS 3580, 103–114, 2005. doi:10.1007/11523468_9. Authors' paper.